

Full Stack Development (FSD)

Documentation

shopEZ — B2C E-Commerce Marketplace (MERN Stack)

DOCUMENT PHASE:	Project Documentation
VERSION:	1.1
DATE:	18th of june, 2026
PREPARED BY:	JAMPA KAILASH JOHNITH
CLASSIFICATION:	Project Documentation — For Academic Submission

Full Stack Development with
MERN — Project Documentation

1. Introduction

- Project Title

ShopEZ — A Production-Grade Full-Stack MERN E-commerce Platform

- Team Members

JAMPA KAILASH JOHNITH — Lead Full-Stack Developer & Solutions Architect

Role: End-to-end implementation of backend API, database modeling, frontend SPA development, security orchestration, state management, and multi-service integration.

2. Project Overview

- Purpose

ShopEZ is developed to address key operational challenges in the modern e-commerce landscape. For small and medium-sized merchants, it provides a highly customizable and scalable online storefront to showcase products, manage inventory, and track orders. For end-consumers, it offers a fast, intuitive, and secure shopping experience with dynamic catalog filtering, persistent state retention, and multi-gateway payment processing.

- Features

User Management & Authentication: JWT sessions stored in secure HTTP-only cookies, combined with BCrypt password hashing and role-based access checks (Admin vs. Customer).

Dynamic Product Catalog: Pagination, keyword search, and filters (by category, price range, and rating) working without full-page reloads.

Persistent Shopping Cart: Syncs shopping cart items directly with MongoDB to retain user selections across multiple devices.

Secure Checkout Flow: A multi-step checkout wizard integrated with Stripe/Razorpay for secure card and UPI payment processing.

Admin Management Panel: Full CRUD operations for products, sales metrics charts, and status tracking for order fulfillment.

Media Handling: Ephemeral uploads optimized and served via the Cloudinary CDN.

3. Architecture

- Frontend

The client is structured as a single-page application (SPA) using React.js. It features:
Context API: Handles global state for user authentication sessions and shopping cart items.

Axios Interceptors: Manages REST API communication, injecting credentials automatically and centralizing error notifications.

Responsive Styling: CSS variables and HSL palettes optimized for mobile, tablet, and desktop viewports.

- Backend

The server runtime is powered by Node.js and structured using Express.js in a strict Model-View-Controller (MVC) architecture:

Routing: Decoupled endpoints for users, products, and orders.

Middleware: Intercepts requests for authentication (protect), role checking (isAdmin), and global exception handling (errorMiddleware).

- Database

We utilize MongoDB Atlas as our cloud NoSQL data store, modeled and queried using the Mongoose ODM:

Data Integrity: Enforces strict schemas and validates inputs before writing to the collections.

Referential Constraints: Uses Mongoose ObjectId references to connect users, carts, products, reviews, and orders.

External Services MongoDB Atlas (Cloud)

Backend – Node.js API (Render)

Frontend – React SPA (Vercel)

HTTPS REST/JSON

Payment API Upload API

Stripe / Razorpay Cloudinary CDN Mongoose ODM
(Schema + Validation)

MongoDB Atlas

Collections

Express.js Router

(/api/users, /products,
/orders, /cart)

JWT Auth + Admin

Middleware

Controllers

(Business Logic)

React.js v18

(Component-based UI)

Context API

(Auth & Cart State)

Axios

(REST API Client)

FIGURE 3.1 — FULL SYSTEM ARCHITECTURE

```
USER
string _id PK
string name
string email
string password
string role
date createdAt
ORDER
string _id      PK
string user FK
number totalPrice
boolean isPaid
boolean isDelivered
string orderStatus
date createdAt
REVIEW
string _id PK
string user FK
string product FK
number rating
string comment
date createdAt
CART
string _id PK
string user FK
date updatedAt
PRODUCT
string _id PK
string name
number price
int stock
number rating
string category FK
ORDER_ITEM      CART_ITEM
CATEGORY
string _id PK
string name
string slug
string description
places
writes
owns
receives
referenced_in
referenced_in
classifies
contains
contains
```

FIGURE 3.2 — ENTITY RELATIONSHIP DIAGRAM

```
places
1
0..*
owns
1
1
writes
1
0..*
receives
1
0..*
classifies
1
0..*
UserModel
+ObjectId _id
+String name
+String email
+String password
+String role
+Array addresses
+matchPassword(enteredPassword) : Boolean
+pre_save_hook() : void
ProductModel
+ObjectId _id
+String name
+String description
+Number price
+Number stock
+Number rating
+Number numReviews
+ObjectId category
+Array images
+Array reviews
OrderModel
+ObjectId _id
+ObjectId user
+Array orderItems
+Object shippingAddress
+String paymentMethod
+Object paymentResult
+Number itemsPrice
+Number taxPrice
+Number totalPrice
+Boolean isPaid
+Date paidAt
+Boolean isDelivered
+Date deliveredAt
+String orderStatus
CartModel
+ObjectId _id
+ObjectId user
+Array items
+Date updatedAt
CategoryModel
+ObjectId _id
+String name
+String slug
+String description
+String image
ReviewModel
+ObjectId _id
+ObjectId user
+ObjectId product
```

+String name

+Number rating

+String comment

+Date createdAt

FIGURE 3.3 — UML CLASS DIAGRAM: CORE MONGOOSE MODELS

4. Setup Instructions

- Prerequisites

Node.js: v18 LTS or above installed.

MongoDB: A cloud cluster on MongoDB Atlas or local installation (v6.x+).

Package Manager: npm v9.x or above.

External accounts: Stripe account (for API test keys) and Cloudinary account (for media uploads).

- Installation
-

1. Configure Environment Variables:

Create a .env file under the /backend folder with the following variables:

PORT=5000

MONGO_URI=mongodb+srv://<username/>:<password>@<cluster>.mo

JWT_SECRET=your_jwt_secret_key

STRIPE_SECRET_KEY=sk_test_your_stripe_key

CLOUDINARY_CLOUD_NAME=your_cloudinary_cloud_name

CLOUDINARY_API_KEY=your_cloudinary_api_key

CLOUDINARY_API_SECRET=your_cloudinary_api_secret

2. Install Dependencies & Seed Database:

Install server dependencies & seed DB

```
cd server /& npm install /& npm run seed
```

Install client dependencies

```
cd ../frontend /& npm install
```

5. Folder Structure

- Client

frontend/

%% % public/	# Static assets
--------------	-----------------

%% % src/

%% % assets/	# Global CSS and themes
%% % components/	# Reusable presentation parts (Nav
%% % context/	# AuthContext.jsx & CartContext.js
%% % pages/	# View routes (Home, ProductDetail
%% % App.jsx	# React routing setup
%% % main.jsx	# DOM mounting entry point

- Server

backend/

%% % config/	# db.js (Mongoose connection pool
%% % controllers/	# User, product, and order control
%% % middleware/	# authMiddleware.js & errorMiddlew
%% % models/	# Mongoose schema definitions (Use
%% % routes/	# Express REST endpoint routes
%% % src/utls/seed.js	# Database seeding script
%% % app.js	# Application entry file

6. Running the Application

To run the application locally in development mode, start both the backend API server and frontend development server:

Backend Server: Run in the /backend directory:

```
npm start
```

(Default port: `http://localhost:5000`)

Frontend Server: Run in the /frontend directory:

```
npm start
```

(Default port: `http://localhost:5173`)

7. API Documentation

Method	Endpoint	Auth Required	Description
POST	/api/auth/register	No	Register a new user account (customer, seller)
POST	/api/auth/login	No	Authenticate user & issue JWT in HTTP-only cookie
POST	/api/auth/logout	Yes	Invalidate user session & clear cookies
GET	/api/auth/profile	Yes	Retrieve authenticated user profile details
PUT	/api/auth/profile	Yes	Update user profile data
GET	/api/products	No	Get all products (supports search, categories, pagination, filters)
GET	/api/products/:slug	No	Get detailed specifications of a single product by slug
POST	/api/products	Yes (Admin/Seller)	Add a new product listing
PUT	/api/products/:id	Yes (Admin/Seller)	Modify a product listing by ID
DELETE	/api/products/:id	Yes (Admin/Seller)	Remove a product listing by ID
POST	/api/reviews/:productId	Yes (Customer)	Add star rating and review comments to a product
POST	/api/orders	Yes (Customer)	Place a new order (COD, Stripe, or Razorpay)
GET	/api/orders/myorders	Yes (Customer)	Retrieve personal order history
GET	/api/orders/:id	Yes	Retrieve order details by ID
PUT	/api/orders/:id/pay	Yes (Customer)	Process Stripe payment and set paid status
PUT	/api/orders/:id/deliver	Yes (Admin)	Mark order status as delivered
GET	/api/orders	Yes (Admin)	View all customer orders

8. Authentication

Authentication on the ShopEZ platform is stateless and utilizes JSON Web Tokens (JWT).

Token Issuance: When a user registers or logs in, the server generates a token signed with the `JWT_SECRET`.

Cookie Storage: The token is written directly to the response header as an HTTP-only, `SameSite=Strict` cookie. This prevents cross-site scripting (XSS) extraction and cross-site request forgery (CSRF) vulnerabilities.

Authorization: Protected routes use the JWT validation middleware:

```
const protect = async (req, res, next) => {  
  const token = req.cookies.jwt;  
  if (!token) return res.status(401).json({ message: 'Not au  
  const decoded = jwt.verify(token, process.env.JWT_SECRET);  
  req.user = await User.findById(decoded.userId).select('-pa  
  next();  
};
```

9. User Interface

The following screenshots showcase different UI features of the ShopEZ application:

FIGURE 9.1 — SHOPEZ HOMEPAGE AND PRODUCT DISCOVERY GRID

FIGURE 9.2 — PRODUCT DETAIL PAGE WITH RATING & REVIEW FORM

FIGURE 9.3 — SHOPPING CART SUMMARY AND PRICE ESTIMATION

10. Testing

The testing strategy for ShopEZ ensures reliability and correctness across multiple levels:

Unit Testing: Testing of utilities, validation rules, and context status reducers (Jest, RTL).

Integration Testing: Running API request/response assertions using Postman and Newman collections to verify route security and status codes.

User Acceptance Testing (UAT): Execution of 16 defined manual UAT scripts covering complete user flow (Registration !' Search !' Cart !' Checkout !' Payment !' Admin Inventory verification).

11. Screenshots or Demo

The platform is deployed online in a decoupled cloud environment. You can access the application and review the administrative features using the following credentials:

Resource	URL / Details
Live Storefront	https://e-commerce-application-neon-five.vercel.app/
Backend API	
URL	https://shopez-api-c30e.onrender.com
Customer Account	
	customer@shopez.com / password123
Admin Account	admin@shopez.com / password123

FIGURE 11.1 — ADMIN PORTAL: PRODUCT CRUD AND INVENTORY

ANALYTICS

FIGURE 11.2 — CUSTOMER LOGIN INTERFACE

FIGURE 11.3 — CUSTOMER ACCOUNT REGISTRATION INTERFACE

12. Known Issues

Issue ID	Description	Mitigation / Plan
KI-01	Cart state may desync on concurrent multi-device sessions.	Server-side updatedAt timestamp comparison on cart save.
KI-02	Render free-tier cold-start causes 30-60s initial delay.	Upgrade to paid tier for production hosting or use keep-alive pings.
KI-03	No email verification on user registration.	Add Nodemailer-based verification flow in future release.

13. Future Enhancements

Priority	Enhancement	Description
Medium	Email Alerts	Nodemailer order dispatch and invoice confirmations.
Low	Wishlists	Saved items library for customer accounts (implemented in routes).
Future	AI Recommendations	Item-based collaborative filtering suggestions for products.
Future	React Native Companion	Mobile application utilizing the existing REST API endpoint structure.

